Secretaría de Educación

DIRECCIÓN GENERAL DE UNIVERSIDADES TECNOLÓGICAS y POLITÉCNICAS

Universidad Tecnológica de Tehuacán

# UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

## INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE


**Alumna:**
**Concepción Guadalupe Paniagua González**


**Asignatura:**
**Desarrollo Web Profesional**

**Docente:**
**José Miguel Carrera Pacheco**

**Octavo Cuatrimestre Grupo “A”**

# S - 6. Eventos, estados y control por teclado

**Event-Driven UI (Arquitectura Dirigida por Eventos)**

Es un paradigma donde el flujo del sistema no sigue un orden lineal, sino que reacciona a cambios de estado o acciones llamadas "eventos". En este modelo, los componentes están desacoplados; un emisor genera una señal y uno o varios suscriptores reaccionan a ella sin que tengan que estar conectados directamente.

React utiliza **Synthetic Events**, que son envoltorios para que los eventos del navegador funcionen igual en cualquier plataforma. Para evitar el "Prop Drilling" (pasar funciones por muchos niveles), se puede implementar un sistema donde los componentes despachen eventos globales que otros escuchan mediante hooks personalizados, manteniendo la comunicación limpia.

Importancia:

- Permite que la interfaz responda en tiempo real a las acciones del usuario.
- Facilita la escalabilidad del sistema, ya que puedes añadir o quitar componentes sin romper la lógica principal.
- Reduce la complejidad del código al evitar cadenas de callbacks infinitas.

Ejemplo: Un sistema de notificaciones global donde, al completar una compra en un componente profundo, se dispara un evento de "éxito" que el componente de la barra de navegación escucha para actualizar un contador, sin que ambos componentes se conozcan entre sí.

**Eventos de teclado en React (onKeyDown, onKeyUp)**

Se trata de capturar cada vez que un usuario interactúa físicamente con su teclado dentro de nuestra aplicación. No se trata solo de escribir texto, sino de reconocer acciones específicas: onKeyDown se activa justo cuando la tecla baja, mientras que onKeyUp espera a que el usuario la suelte. Es la forma en que el software "siente" el hardware.

En React, no usamos el método tradicional de buscar elementos en el DOM para ponerles un escucha. En su lugar, lo manejamos de forma declarativa dentro del JSX, asignando funciones a propiedades como onKeyDown. Lo mejor es que React nos entrega un objeto de evento "sintético" que es muy sencillo de leer. Por ejemplo, podemos acceder a event.key para saber si la persona presionó "Enter", "Escape" o cualquier otra tecla, y así decidir qué lógica ejecutar en ese preciso momento.

Importancia:

- Inclusión: Es la única manera en que muchas personas con discapacidades motoras pueden navegar por la web sin depender de un ratón.
- Experiencia de Usuario (UX): Los usuarios avanzados agradecen los atajos (shortcuts) que les permiten hacer tareas complejas sin despegar las manos del teclado.
- Control total: En componentes como formularios o menús desplegables, los eventos de teclado nos permiten validar datos al instante o cerrar ventanas de forma intuitiva.

Ejemplo:Imagina que estás desarrollando un buscador. En lugar de obligar al usuario a buscar el botón de "lupa" con el mouse, puedes programar un manejador que detecte cuando event.key === 'Enter' y dispare la búsqueda automáticamente. También es lo que usamos para que, al presionar "Escape", un modal molesto desaparezca de la pantalla de inmediato, mejorando la fluidez de la interfaz.

**Estados Accesibles (ARIA, Focus Management)**

Son prácticas y atributos que aseguran que la interfaz sea entendible por tecnologías asistivas (como lectores de pantalla) y navegable mediante teclado. ARIA proporciona el contexto semántico que el HTML básico no tiene.

Función en React:

- ARIA: Se usan atributos como aria-expanded (para indicar si algo está abierto) o aria-labelledby (para dar nombre a elementos). En React, estos atributos se escriben igual que en HTML.
- Focus Management: Se utiliza para controlar qué elemento tiene el "foco" del teclado en cada momento. En React, se suelen usar refs para mover el foco programáticamente a un elemento específico tras una acción.

Importancia:

- Garantiza que personas con discapacidades visuales o motoras puedan usar la aplicación.
- Evita que el usuario "se pierda" al navegar (por ejemplo, si el foco desaparece al cerrar un menú).
- Cumple con los estándares internacionales de accesibilidad web.

Ejemplo: Al abrir un menú lateral (sidebar), se debe cambiar el estado de aria-expanded a true y mover automáticamente el foco al primer enlace del menú. Al

cerrarlo, el foco debe regresar por obligación al botón que lo abrió originalmente para no desorientar al usuario.

# Bibliografía

*ARIA Authoring Practices Guide (APG)*. (s/f). Www.w3.org. https://www.w3.org/WAI/ARIA/apg/

*Event-driven architecture for clean react component communication*. (2024, diciembre 4). DEV Community. https://dev.to/nicolalc/event-driven-architecture-for-clean-react-component-communication-fph

*Event-driven architecture(EDA)*. (2024, enero 31). GeeksforGeeks. https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/

*Responding to events*. (s/f). React.dev. https://react.dev/learn/responding-to-events